

PyBED: A Composable and Reproducible Interface for Bayesian Experimental Design

PyBED contributors

Version 0.1.0 – September 2026

Abstract

Bayesian experimental design (BED) methods often share scientific problems but not their data abstractions, simulator interfaces, random-number conventions, metrics, or experiment records. This makes apparently direct comparisons difficult to audit. PyBED is a compact Python package that standardizes these common layers while leaving the method itself replaceable. Its central object composes a prior, simulator, space specifications, and optional design, inference, prediction, and visualization components. Explicit seed derivation guarantees the same ordered latent parameters within a synthetic epoch across competing policies. Finite generated epochs can be saved and reloaded for posterior-only training. Reference environments cover source location, MNIST image discovery, initial-field inference in an advection-diffusion PDE, stochastic pendulum identification, CES preferences, and a pure-death process. This report motivates the design, defines its contracts, documents the numerical models, and explains the included single-file ActionBED example and validation suite.

Contents

1	Motivation	3
2	Architecture	3
2.1	The BED value object	3
2.2	The batch interchange object	4
2.3	Module boundaries	4
3	Randomness and the meaning of an epoch	5
3.1	Named random streams	5
3.2	Finite synthetic epochs and offline data	5
3.3	Non-stationarity	5
4	Reference environments and simulators	6
4.1	Location finding	6
4.2	MNIST image discovery	6
4.3	Advection-diffusion initial-field inference	6
4.4	Stochastic and behavioral models	7
5	Metrics and comparisons	7

6	ActionBED example	8
6.1	Model	8
6.2	Workflow	8
7	Experiment records	8
8	Method integration	9
9	Validation strategy	9
10	Limitations and roadmap	9
11	Conclusion	10

1 Motivation

A sequential BED problem has an unknown parameter θ , a design ξ_t , an outcome y_t , and history

$$h_t = \{(\xi_i, y_i)\}_{i=1}^t. \quad (1)$$

A policy π maps the history to a distribution over the next design, while an inference method represents $p(\theta \mid h_t)$ or a target derived from it. This abstract structure appears in amortized likelihood-based design, likelihood-free design, joint acquisition and inference, and particle-transport methods. DAD amortizes a non-myopic policy using sequential prior contrastive estimation [1]; iDAD replaces an explicit likelihood with mutual-information estimators [2]; ALINE jointly represents inference and targeted acquisition with structured attention [3]; and JADAI trains a policy and high-capacity posterior estimator end to end [4].

Despite this shared mathematical core, published implementations commonly differ in simulator constants, tensor order, padding, episode generation, seed reuse, metric reduction, and run book-keeping. The result is avoidable uncertainty: a comparison may measure a dataloader or simulator difference instead of a policy difference. PyBED therefore standardizes the experimental substrate, not the learning algorithm.

The package is guided by four constraints:

1. **Small interface.** The public verbs are `simulate`, `design`, `infer`, and `predict`.
2. **Composition over inheritance.** Components are callables that can be replaced on an immutable environment value.
3. **Explicit randomness.** An experiment is reproducible from named seed streams rather than hidden mutable generator state.
4. **Method neutrality.** PyBED does not require a particular posterior, policy, objective, optimizer, or trainer.

2 Architecture

2.1 The BED value object

The class `BED` is an immutable value containing the following fields.

Field	Contract
<code>prior</code>	Object with <code>sample(shape, generator=...)</code> and optional <code>log_prob</code> ; may also be a context-dependent callable.
<code>sim</code>	Callable from parameter and design to an observation; receives explicit generator and accepted context arguments.
<code>specs</code>	Event-shape and optional bound declarations for <code>theta</code> , <code>design</code> , and <code>obs</code> .
<code>policy</code>	Optional callable mapping a history to the next design.
<code>posterior</code>	Optional callable mapping a history and target context to an inference object.
<code>predictor</code>	Optional callable mapping a query and history to a prediction.
<code>visualizers</code>	Named plot callables local to the problem or user.
<code>cfg</code>	Readable numerical configuration and provenance.
<code>target</code>	Optional callable defining a label, parameter subset, image, or predictive target from θ .

Event shapes deliberately exclude batch and history axes. Thus `Spec((2,))` accepts both a single two-dimensional design and a batch of them. The summary returned by `repr(env)` exposes these contracts before any model is constructed.

Composition uses `with_components`:

```
method_env = env.with_components(
    simulate=custom_simulator,
    design=policy.design,
    infer=posterior.infer,
    predict=predictor.predict,
)
```

The original environment remains unchanged. Callable signatures are inspected, and only compatible context arguments are forwarded. This permits a two-argument NumPy simulator, a PyTorch module that consumes masks, or a richer callable receiving `epoch`, `time`, and `target` to occupy the same slot.

2.2 The batch interchange object

`Batch` contains arrays for θ , designs, observations, a valid-history mask, an arbitrary target, context, and metadata. Fixed and variable-length histories therefore share one representation:

$$\theta \in \mathbb{R}^{B \times d_\theta}, \quad (2)$$

$$\xi \in \mathbb{R}^{B \times T \times d_\xi}, \quad (3)$$

$$y \in \mathbb{R}^{B \times T \times d_y}, \quad (4)$$

$$m \in \{0, 1\}^{B \times T}. \quad (5)$$

The event dimensions may themselves be structured; for example, an initial field has shape $1 \times H \times W$. The mask is general enough for padded history encoders. ALINE-like target-specific attention remains a responsibility of the model, while the requested target subset can be carried in `target`. This separation prevents a particular Transformer mask from becoming an environment assumption.

2.3 Module boundaries

Module	Responsibility
core	Environment, batch, spaces, priors, component dispatch, registry.
sim	Batched reference models, differentiable PDE stepping, and SDE integration.
envs	Configurable factories with paper-related defaults.
data	Seed derivation, synthetic epochs, serialization, deterministic loaders.
metrics	Posterior, predictive, calibration, and information scores.
vs	White-grid publication graphics with large labels and ticks.
exp	Run folders, JSONL diagnostics, checkpoints, training, comparisons.
backends	Explicit conversion among PyTorch, NumPy, and optional JAX arrays.

This structure is intentionally conventional. There is no internal plugin hierarchy, configuration framework, event bus, or tracking service.

3 Randomness and the meaning of an epoch

3.1 Named random streams

A single sequential generator is fragile: adding one diagnostic draw changes all later training data. `SeedBank` instead hashes

$$s_{n,e,t} = H(s_0, n, e, t), \quad (6)$$

where s_0 is the root seed, n is a human-readable namespace, e is an epoch, and t is a step or other integer index. The BLAKE2b digest is mapped into PyTorch’s generator seed range. Parameter sampling, design exploration, simulator noise, dataloader shuffling, and worker state use separate namespaces.

For environment E , the ordered truth tensor in epoch e is

$$\Theta_e = \text{sample}(p_E, N; H(s_0, \text{“E:theta”}, e)). \quad (7)$$

It does not depend on the policy name or the order in which policies are evaluated. Accordingly, for any policies π_a and π_b generated from one `EpochStream`,

$$\Theta_e^{(a)} = \Theta_e^{(b)} \quad (8)$$

exactly, not merely in distribution. Common simulator random numbers can also be enabled. Their transformations may differ because the policies choose different designs, but the underlying random draw at each episode and step is coupled.

3.2 Finite synthetic epochs and offline data

An online epoch is defined as N newly materialized latent truths and their complete rollouts. It is a bookkeeping and diagnostic unit, not an assertion about an infinite population. The generated object implements the ordinary PyTorch dataset protocol. Saving it stores tensors plus a JSON sidecar containing the format version, shapes, and experiment metadata.

This unifies two training modes:

- In *online* training, each yielded loader is constructed from a newly generated finite epoch.
- In *posterior-only* training, the loader traverses an already saved dataset; a complete traversal is one conventional epoch.

The generic `exp.train` loop accepts either sequence. Data can therefore be generated once for fast PSST-like posterior training, archived, and replayed by multiple inference methods without simulator cost.

3.3 Non-stationarity

The prior slot may be a callable returning a prior from context. A simulator may also accept a physical time, regime identifier, or exogenous state. Therefore a changing parameter law or forward model does not require a new core class. Explicit context is written into batches and can be included in run configuration. Reproducibility still requires the schedule itself to be deterministic from recorded context and seeds.

4 Reference environments and simulators

4.1 Location finding

For K sources at positions $\theta_k \in \mathbb{R}^d$ and sensor design $\xi \in \mathbb{R}^d$, the noise-free intensity is

$$\mu(\theta, \xi) = b + \sum_{k=1}^K \frac{\alpha_k}{m + \|\theta_k - \xi\|_2^2}. \quad (9)$$

The default observation follows the modern ALINE/JADAI form

$$y = \log \mu(\theta, \xi) + \sigma \epsilon, \quad \epsilon \sim \mathcal{N}(0, 1), \quad (10)$$

with $b = 0.1$, $m = 10^{-4}$, $\alpha_k = 1$, and $\sigma = 0.5$ [3, 4]. All constants, dimensionalities, source counts, bounds, and log transformation are configurable. Because noise is reparameterized, gradients flow from a posterior loss through y and ξ into a continuous policy.

Multiple sources are exchangeable. The metric module therefore provides exact permutation-invariant set MSE for the small K used in these benchmarks.

4.2 MNIST image discovery

The unknown is an image $\theta \in [0, 1]^{C \times H \times W}$ and the design is a continuous mask center $\xi \in [0, 1]^2$. The smooth bivariate-logistic mask is a product of two soft rectangular windows. For pixel coordinate $x = (x_1, x_2)$,

$$M_{\xi, s, h}(x) = [S(x_1 - \xi_1 + h; s) + S(\xi_1 + h - x_1; s) - 1] \quad (11)$$

$$\times [S(x_2 - \xi_2 + h; s) + S(\xi_2 + h - x_2; s) - 1], \quad (12)$$

where $S(a; s) = (1 + \exp(-a/s))^{-1}$. The observation is

$$y(x) = \text{clip}_{[0,1]} \{\theta(x) M_{\xi, s, h}(x) + \eta(x)\}, \quad \eta(x) \sim \mathcal{U}(0, \sigma). \quad (13)$$

This follows the image-discovery setup documented by JADAI [4]. Arrays are supplied directly or loaded lazily through the optional torchvision extra. No network access occurs by default. If labels are provided, they become available as classification targets; otherwise the natural target is image reconstruction.

4.3 Advection-diffusion initial-field inference

Alexanderian et al. consider an unknown initial condition m propagated by

$$u_t - \kappa \Delta u + v \cdot \nabla u = 0 \quad \text{in } D \times (0, T), \quad (14)$$

$$u(\cdot, 0) = m, \quad (15)$$

then observed at fixed sensor locations and discrete times [5]. Thus a sensor returns a time series, while the parameter is a spatial field. PyBED preserves exactly this parameter-design-observation abstraction.

Its compact reference solver generalizes the equation to an optional linear reaction term ru and uses:

1. a differentiable semi-Lagrangian bilinear resampling step for advection;
2. a five-point explicit Laplacian for diffusion;

3. bilinear sensor sampling at user-specified normalized coordinates;
4. additive reparameterized Gaussian observation noise.

The solver checks the explicit diffusion stability ratio $\kappa\Delta t/\Delta x^2 \leq 0.24$. It accepts a constant velocity, a spatial velocity tensor, or a time-dependent callable. It is independent of the environment factory and can be reused in other PDE inverse problems.

This is deliberately not a numerical reproduction of the paper’s obstacle geometry, finite-element discretization, or Navier-Stokes-derived velocity field. Those require a specialized PDE package and mesh data. A researcher can replace `sim` with such a solver while retaining PyBED’s episode and comparison protocol.

4.4 Stochastic and behavioral models

The Euler-Maruyama routine integrates

$$X_{n+1} = X_n + a(X_n, t_n)\Delta t + b(X_n, t_n)\sqrt{\Delta t}\epsilon_n. \quad (16)$$

The pendulum environment uses state $(q, \dot{q})$ and unknown gravity g and damping c :

$$dq = \dot{q} dt, \quad d\dot{q} = (-g \sin q - c\dot{q})dt + \sigma dW_t. \quad (17)$$

The design selects its initial angle and velocity, and the observation is a retained angle time series. The module additionally includes a CES preference experiment and a likelihood-free pure-death process, covering representative behavioral and epidemiological examples used in amortized design work [1, 2].

5 Metrics and comparisons

PyBED keeps metrics in a separate module so environment visualization does not imply a particular scientific claim. Let $\hat{\theta}$, θ^* , and posterior particles $\{z_s\}_{s=1}^S$ be given. Included diagnostics are:

$$\text{MSE} = d^{-1}\|\hat{\theta} - \theta^*\|_2^2, \quad (18)$$

$$\log\text{MSE} = \log(\text{MSE} + \epsilon), \quad (19)$$

$$\text{ES} = \frac{1}{S} \sum_s \|z_s - \theta^*\| - \frac{1}{2S^2} \sum_{s,r} \|z_s - z_r\|. \quad (20)$$

Negative log score, RMSE, interval coverage, posterior-rank calibration error, posterior spread, and permutation-invariant source MSE are also provided.

For tractable likelihoods, sequential prior contrastive estimation accepts the accumulated history log-likelihood under a generating draw θ_0 and L prior contrastives:

$$\hat{I}_{\text{sPCE}} = \frac{1}{B} \sum_{b=1}^B \left[\ell_b(\theta_0) - \log \left(\frac{1}{L+1} \sum_{j=0}^L \exp \ell_b(\theta_j) \right) \right]. \quad (21)$$

This is a lower bound on sequential expected information gain under the usual sPCE sampling construction [1]. The API requires log-likelihoods rather than guessing them from simulator samples, which keeps likelihood-free and likelihood-based evaluation scientifically distinct.

`exp.compare` evaluates a name-to-policy mapping from one `EpochStream`. It asserts exact equality of the latent truth tensor, returns metric dictionaries, and retains every trajectory. A paper result can therefore be traced back to designs, observations, targets, masks, and seed metadata.

6 ActionBED example

The one-file example in `examples/actionbed/actionbed.py` exercises the public surface rather than adding privileged library hooks. It is an integration baseline, not a reproduction of a named paper.

6.1 Model

Each $(\xi_t, y_t, t/T)$ token is embedded by a multilayer perceptron. A masked mean and masked maximum produce a permutation-aware fixed-width history representation. An empty-history embedding is learned. Two heads consume the representation:

- a diagonal Gaussian posterior $q_\psi(\theta \mid h_t)$;
- a Gaussian latent action distribution whose reparameterized sample is mapped through a sigmoid into the design bounds.

At every step, gradients pass through the action, location simulator, new outcome, history encoder, and posterior. The joint prefix objective is

$$\mathcal{L}(\phi, \psi) = \frac{1}{T} \sum_{t=1}^T \mathbb{E}[-\log q_\psi(\theta^* \mid h_t)] - \lambda \mathcal{H}[\pi_\phi(\cdot \mid h_T)]. \quad (22)$$

Thus both inference and acquisition parameters are trained from posterior quality. The entropy term is a small collapse-prevention regularizer. The example uses gradient clipping and AdamW, but neither is part of the environment contract.

6.2 Workflow

The file follows a notebook narrative:

1. construct, print, and visualize the environment;
2. generate an intuitive random trajectory;
3. define the model in the same file;
4. train over explicitly materialized truth epochs;
5. attach `design`, `infer`, and `predict`;
6. compare learned and random policies on common truths and noise;
7. save log-scale loss curves, trajectories, datasets, checkpoints, and summary.

The environment variable `PYBED_QUICK=1` selects a two-epoch smoke configuration for continuous integration.

7 Experiment records

`Run.create` records normalized configuration, its SHA-256 digest, creation time, Python and PyTorch versions, platform, CUDA version, root seed, and Git commit when available. Metrics are append-only JSON Lines. A checkpoint contains model and optimizer state plus Python, NumPy, CPU Torch, and all CUDA RNG states. Restoring the checkpoint can therefore restore stochastic continuation as well as parameters.

The folder layout is intentionally transparent:

```
runs/<experiment>/<timestamp>-<config-hash>/
  config.json
  meta.json
```



```
metrics.jsonl
summary.json
checkpoints/*.pt
figures/*.png
data/*.pt
data/*.pt.json
```

There is no database, network service, or required tracking account.

8 Method integration

The framework admits the major method families without method-specific branches:

- **DAD.** Attach its policy, generate histories, and compute sPCE from its tractable likelihood evaluations.
- **iDAD.** Use the same histories while optimizing InfoNCE or NWJ in the external model; no environment likelihood is required.
- **JADAI.** Attach both policy and posterior. High-dimensional image targets remain arrays in the batch.
- **ALINE.** Use `mask` for valid context and `target` for selected parameter/predictive targets; its internal structured attention mask stays within the model.
- **PSST-like transport.** Construct `EmpiricalPrior` from a cloud, or put a different cloud in each batch context. Save a synthetic dataset once for fast non-sequential posterior training.

The JAX extra converts loader batches explicitly, with DLPack when possible. Built-in simulators are PyTorch because the requested reference implementation is PyTorch-first. PyBED does not pretend that automatic translation of a PDE solver preserves floating point behavior. A pure-JAX or NumPy callable can be attached when that backend is required.

9 Validation strategy

The test suite emphasizes contracts likely to invalidate a benchmark:

1. event-shape and bound checks reject malformed calls;
2. component replacement leaves the source environment unchanged;
3. identical epoch requests reproduce truths, designs, observations, and loader order;
4. competing policies receive bit-identical ordered truths;
5. saved episodes round trip with metadata and masks;
6. simulators return declared shapes and gradients for differentiable designs;
7. PDE stability checks fail early;
8. sPCE, proper scores, source matching, and posterior summaries are tested on analytically simple cases;
9. the ActionBED quick configuration runs end to end in a subprocess.

Package build, byte-code compilation, tests, a random-policy example, and the single-file ActionBED smoke run form the release gate. The technical report is also compiled and rendered to images for layout inspection.

10 Limitations and roadmap

Version 0.1 intentionally avoids a universal model abstraction. Some remaining work is scientific rather than infrastructural:

- validate paper-exact environment presets and publish frozen reference datasets;
- provide independently reviewed reproduction adapters for DAD, iDAD, ALINE, JADAI, and PSST rather than simplified namesake baselines;
- add distributed episode generation while retaining namespace-stable seeds;
- add discrete, constrained, and set-valued design-space helpers;
- implement paper-scale PDE meshes as optional integrations;
- add pure-JAX reference simulators only where equivalence tests can be stated;
- publish benchmark cards specifying likelihood, prior, units, constants, precision, target, horizon, and metric estimator uncertainty.

Two cautions are central. First, common random numbers reduce variance but do not make policy-induced data identical; designs intentionally change the conditional observation distribution. Second, a shared environment is necessary but not sufficient for fair comparison. Training budget, architecture size, contrastive count, posterior sample count, evaluation seeds, and uncertainty estimates must also be reported.

11 Conclusion

PyBED turns the repeated infrastructure of Bayesian experimental design into a small, inspectable package. The environment is a composition of scientific callables; the batch makes histories and targets portable; the epoch stream makes synthetic training and cross-policy comparison reproducible; and metrics, plots, and local run records remain usable independently. This creates a practical boundary between a benchmark’s scientific problem and a method’s inductive bias, enabling researchers to replace the latter without accidentally replacing the former.

References

- [1] A. Foster, D. R. Ivanova, I. Malik, and T. Rainforth. Deep Adaptive Design: Amortizing Sequential Bayesian Experimental Design. *Proceedings of ICML*, 2021. <https://proceedings.mlr.press/v139/foster21a.html>.
- [2] D. R. Ivanova, A. Foster, S. Kleinegesse, M. U. Gutmann, and T. Rainforth. Implicit Deep Adaptive Design: Policy-Based Experimental Design without Likelihoods. *Advances in Neural Information Processing Systems*, 2021. <https://proceedings.neurips.cc/paper/2021/hash/357a6fdf7642bf815a88822c447d9dc4-Abstract.html>.
- [3] D. Huang, X. Wen, A. Bharti, S. Kaski, and L. Acerbi. ALINE: Joint Amortization for Bayesian Inference and Active Data Acquisition. arXiv:2506.07259, 2025. <https://arxiv.org/abs/2506.07259>.
- [4] N. Bracher, L. Kuehmichel, D. R. Ivanova, X. Intes, P.-C. Buerkner, and S. T. Radev. JADAI: Jointly Amortizing Adaptive Design and Bayesian Inference. arXiv:2512.22999, 2025. <https://arxiv.org/abs/2512.22999>.
- [5] A. Alexanderian, N. Petra, G. Stadler, and O. Ghattas. A-optimal design of experiments for infinite-dimensional Bayesian linear inverse problems with regularized ℓ_0 -sparsification. *SIAM Journal on Scientific Computing*, 36(5):A2122–A2148, 2014. <https://doi.org/10.1137/130933381>.